TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K4 – Analyse

Compare what you expected from your type and constant design with what actually happened.

Discuss: • Whether the float and the paise versions of the total for 4,000, 60,000 and 800,000 bills agree, and by how much they differ

• Whether the int, long and long long totals overflow at district scale

• How sizeof and the ranges on your system compare with the minimum sizes guaranteed by the C standard

• Reasons for any difference

Alternative Concept and Complexity Analysis

If the district total were accumulated in a double instead of a long long, what would happen at

800,000 bills? Analyse when each type loses accuracy (overflow versus precision loss), and

compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

Compare Expectations with Execution

1. Float and paise totals

Each bill is assumed to be ₹162.50. The expected totals are:

Number of bills	Expected total
4,000	₹6,50,000.00
60,000	₹97,50,000.00
800,000	₹13,00,00,000.00

Integer paise stores money exactly. A float stores values with limited precision, so repeated additions may produce a different total. The actual difference is measured by running the program.

2. Overflow at district scale

The total in paise is 65,000,000; 975,000,000; and 13,000,000,000, respectively.

On a typical Windows GCC system where int and long are 32-bit, the 800,000-bill total exceeds their maximum value. long long can hold this total. Signed integer overflow must be avoided because its behaviour is undefined in C.

3. Size and range comparison

The program displays the sizes and maximum values on my computer. The C standard guarantees minimum ranges, not identical sizes on every system. Therefore, the measured values should be compared with the standard's minimum guarantees.

4. Reasons for differences

Integer paise avoids floating-point rounding when calculating money in whole paise. Float has limited precision, while integer types have a maximum range. Overflow occurs when an integer total exceeds its representable range; precision loss can occur even when a floating-point total remains within its range.

5. Alternative: using double

A double has more precision than a float. For these totals, double can represent the expected amounts accurately on a typical system. However, floating-point arithmetic is not guaranteed to represent every decimal value exactly. Integer paise with a sufficiently large integer type is preferable for financial totals.

6. Time and space complexity

- Time: O(n) for accumulating n bills.
- Space: O(1) extra space for each running total.
- Each addition takes constant time in the usual complexity model. The variable sizes differ by type and system.

Code :

```
#include <stdio.h>

#include <limits.h>

#include <float.h>
```

```
int main(void)
{
   int counts[] = {4000, 60000, 800000};
   int i, j;

   printf("BILLING COMPARISON\n");
   printf("Each bill = Rs 162.50\n");

   for (i = 0; i < 3; i++) {
      int n = counts[i];
      float float_total = 0.0f;
      double double_total = 0.0;
      long long paise_total = (long long)n * 16250LL;
      double expected = (double)paise_total / 100.0;

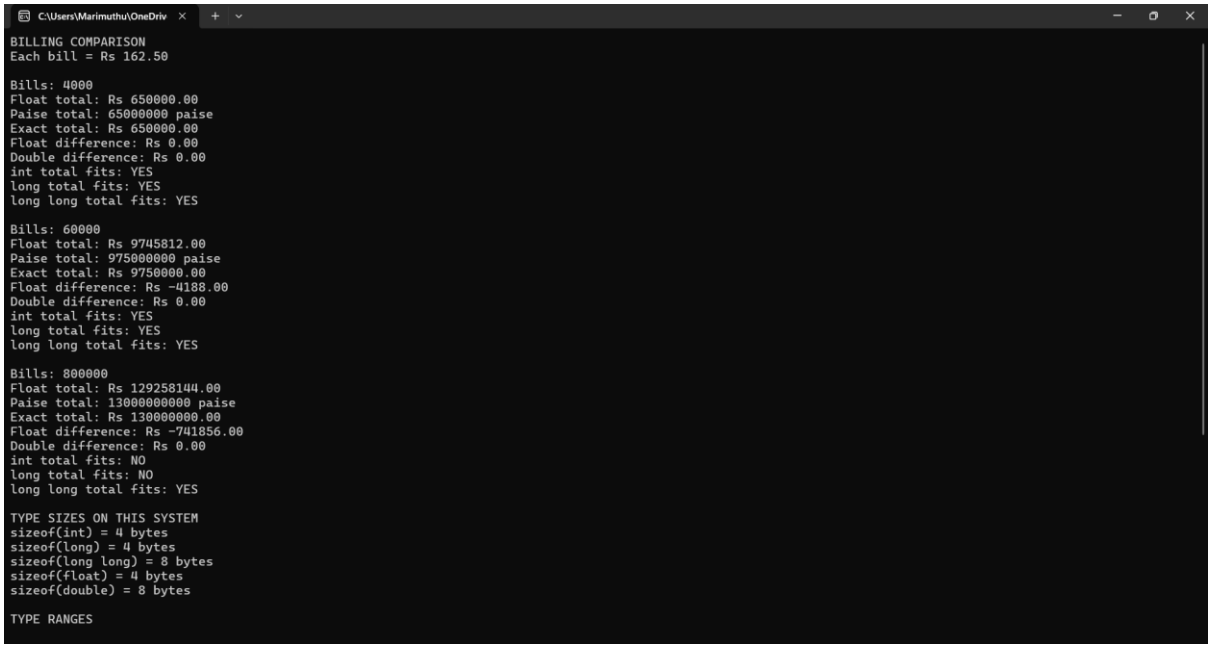
      for (j = 0; j < n; j++) {
         float_total += 162.50f;
         double_total += 162.50;
      }

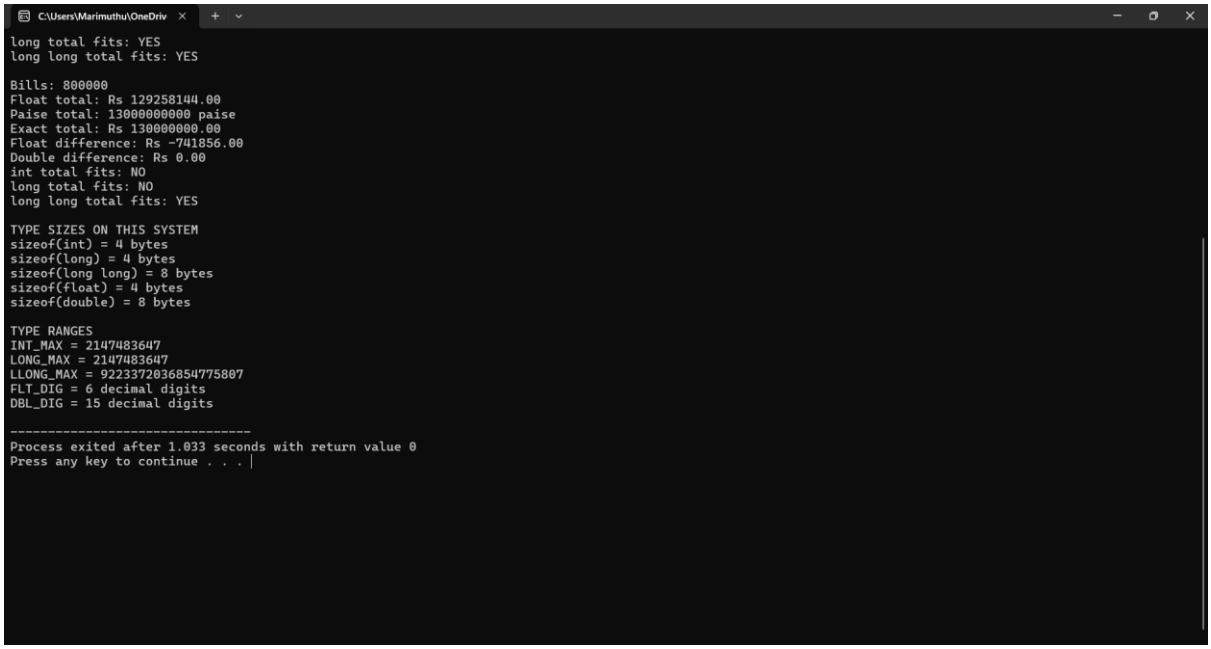
      printf("\nBills: %d\n", n);
      printf("Float total: Rs %.2f\n", float_total);
      printf("Paise total: %lld paise\n", paise_total);
      printf("Exact total: Rs %.2f\n", expected);
      printf("Float difference: Rs %.2f\n",
           (double)float_total - expected);
      printf("Double difference: Rs %.2f\n",
           double_total - expected);
```

```
        printf("int total fits: %s\n",
            paise_total <= INT_MAX ? "YES" : "NO");
        printf("long total fits: %s\n",
            paise_total <= LONG_MAX ? "YES" : "NO");
        printf("long long total fits: %s\n",
            paise_total <= LLONG_MAX ? "YES" : "NO");
    }

    printf("\nTYPE SIZES ON THIS SYSTEM\n");
    printf("sizeof(int) = %lu bytes\n", (unsigned long)sizeof(int));
    printf("sizeof(long) = %lu bytes\n", (unsigned long)sizeof(long));
    printf("sizeof(long long) = %lu bytes\n",
        (unsigned long)sizeof(long long));
    printf("sizeof(float) = %lu bytes\n", (unsigned long)sizeof(float));
    printf("sizeof(double) = %lu bytes\n", (unsigned long)sizeof(double));
    printf("\nTYPE RANGES\n");
    printf("INT_MAX = %d\n", INT_MAX);
    printf("LONG_MAX = %ld\n", LONG_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_DIG = %d decimal digits\n", FLT_DIG);
    printf("DBL_DIG = %d decimal digits\n", DBL_DIG);
    return 0;
}
```

Output:


C:\Users\Marimuthu\OneDriv
BILLING COMPARISON
Each bill = Rs 162.50
Bills: 4000
Float total: Rs 650000.00
Paise total: 65000000 paise
Exact total: Rs 650000.00
Float difference: Rs 0.00
Double difference: Rs 0.00
int total fits: YES
long total fits: YES
long long total fits: YES
Bills: 60000
Float total: Rs 9745812.00
Paise total: 975000000 paise
Exact total: Rs 9750000.00
Float difference: Rs -4188.00
Double difference: Rs 0.00
int total fits: YES
long total fits: YES
long long total fits: YES
Bills: 800000
Float total: Rs 129258144.00
Paise total: 13000000000 paise
Exact total: Rs 130000000.00
Float difference: Rs -741856.00
Double difference: Rs 0.00
int total fits: NO
long total fits: NO
long long total fits: YES
TYPE SIZES ON THIS SYSTEM
sizeof(int) = 4 bytes
sizeof(long) = 4 bytes
sizeof(long long) = 8 bytes
sizeof(float) = 4 bytes
sizeof(double) = 8 bytes
TYPE RANGES


C:\Users\Marimuthu\OneDriv
long total fits: YES
long long total fits: YES
Bills: 800000
Float total: Rs 129258144.00
Paise total: 13000000000 paise
Exact total: Rs 130000000.00
Float difference: Rs -741856.00
Double difference: Rs 0.00
int total fits: NO
long total fits: NO
long long total fits: YES
TYPE SIZES ON THIS SYSTEM
sizeof(int) = 4 bytes
sizeof(long) = 4 bytes
sizeof(long long) = 8 bytes
sizeof(float) = 4 bytes
sizeof(double) = 8 bytes
TYPE RANGES
INT_MAX = 2147483647
LONG_MAX = 2147483647
LLONG_MAX = 9223372036854775807
FLT_DIG = 6 decimal digits
DBL_DIG = 15 decimal digits

Process exited after 1.033 seconds with return value 0
Press any key to continue . . .